

Slipst Advanced

Here are some uncategorized advanced features of Slipst that didn’t fit into the tutorial.

Absolute Positioning with up

We know that up can point to an anchor slip, which means to scroll to the top of the anchor slip.

Here #up(<1>) will scroll to the top of the previous slip, which contains the label <1>.

However, sometimes you may want to scroll to the middle of a slip. For example, when there is a looong paragraph...

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae.

We use #up(<long>, dy: 5cm) to scroll to a position 5cm below the top of the slip <long>.

Altering Slips, a.k.a. Replacing Animations

In beamer/polylux/touying, you can have animations by defining multiple versions of a slide, and then revealing them one by one.

In slipst, many of these animations can already be achieved by #pause and #up, but sometimes you may want to have more control, e.g. to have the left half of the content appear first, and then the right half, or to let a figure of tree grow branch by branch.

This is where #alter comes in. Insert #alter(n) to mark the current slip has n versions.

typ

```
1 #pause
2 #alter(3)
```

Inside the slip with #alter(n), you can use #uncover(...) to specify which version(s) of the slip a content belongs to. The syntax of #uncover is inspired by polylux/touying’s, for example:

- #uncover("2") means the content only appears in 2nd version.
- #uncover("1-3") means the content appears in 1st, 2nd, and 3rd versions.
- #uncover("2-") means the content appears in 2nd version and all later versions.
- #uncover(("1", "3")) means the content appears in 1st and 3rd versions, but not in 2nd version

typ

```
1 #alter(3)
2 This slip has 3 versions.
3
4 #uncover("1")[This is the 1st version.]
5 #uncover("2")[This is the 2nd version.]
6 #uncover("3")[This is the 3rd version.]
```

This slip has 3 versions.

This is the 1st version. This is the 2nd version. This is the 3rd version.

As you can see, the content not shown still takes up space. If you want the content to not take up space, you can use #only instead of #uncover.

typ

```
1 #alter(3)
2 This slip has 3 versions.
3
4 #only("1")[This is the 1st version.]
5 #only("2")[This is the 2nd version.]
6 #only("3")[This is the 3rd version.]
```

This slip has 3 versions.

This is the 1st version. This is the 2nd version. This is the 3rd version.

#uncover can also be used in math equations:

$$\begin{aligned} & \text{filter } p \text{ (map } f \text{ (} y : ys \text{))} \\ &= \text{filter } p \text{ (} f \text{ } y : \text{map } f \text{ } ys \text{)} \\ &= \text{if } p \text{ (} f \text{ } y \text{) then } f \text{ } y : \text{filter } p \text{ (map } f \text{ } ys \text{) else filter } p \text{ (map } f \text{ } ys \text{)} \\ &= \text{if } p \text{ (} f \text{ } y \text{) then } f \text{ } y : \text{map } f \text{ (filter (} p \circ f \text{) } ys \text{) else map } f \text{ (filter (} p \circ f \text{) } ys \text{)} \\ &= \text{if } p \text{ (} f \text{ } y \text{) then map } f \text{ (} y : \text{filter (} p \circ f \text{) } ys \text{) else map } f \text{ (filter (} p \circ f \text{) } ys \text{)} \\ &= \text{map } f \text{ (if } p \text{ (} f \text{ } y \text{) then } y : \text{filter (} p \circ f \text{) } ys \text{ else filter (} p \circ f \text{) } ys \text{)} \\ &= \text{map } f \text{ (filter (} p \circ f \text{) (} y : ys \text{))} \end{aligned}$$

And in CeTZ/fletcher diagrams. You need to customize the cover parameter of #uncover to work with diagrams. The cover parameter is a function that is used to cover the content that is not shown. For technical reasons, you also need to set raw: true and wrap the diagram with context to make it work.

typ

```
1 #let only = uncover.with(cover: fletcher.hide, raw: true)
2 #context diagram(
3   node((0, 0), $⊢ A ∧ (B => C)$),
4   only("2-", node((-0.6, 0.7), $⊢ A$)),
5   // ...
6 )
```

